# Formal Logic (SM234614)

## SAT in Z3

Muhammad Syifa'ul Mufid, D.Phil.

**Department of Mathematics**

Institut Teknologi Sepuluh Nopember Surabaya

# Example of a SAT Formula

Consider the formula

$$\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3).$$

- Variables: $x_1, x_2, x_3$
- Operators: $\wedge, \vee, \neg$
- Goal: find a truth assignment that makes all clauses true

# SAT as a Constraint Problem

SAT can be viewed as a constraint satisfaction problem.

$$(x_1 \vee x_2) = \text{True}$$

$$(\neg x_1 \vee x_3) = \text{True}$$

$$(\neg x_2 \vee \neg x_3) = \text{True}$$

**Satisfying Assignment**

A solution is an assignment of Boolean values satisfying all constraints simultaneously.

# What is Z3?

Z3 is an SMT solver developed by Microsoft Research.

- SAT solver: handles Boolean formulas
- SMT solver: extends SAT with theories such as integers, arrays, bit-vectors, etc.
- Can be used from Python
- Suitable for modeling logical and mathematical constraints

# Using Z3 in Google Colab

In Google Colab, install Z3 using:

```
!pip install z3-solver
```

Then import Z3:

```
from z3 import *
```

# Encoding Boolean Variables

We define Boolean variables using `Bool`.

```
from z3 import *

x1 = Bool('x1')
x2 = Bool('x2')
x3 = Bool('x3')
```

These variables can take values:

$$x_i \in \{\text{True}, \text{False}\}.$$

# Encoding the SAT Formula

The formula

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$$

can be written in Z3 as:

```
# Boolean variables
x1 = Bool('x1')
x2 = Bool('x2')
x3 = Bool('x3')

# SAT formula
phi = And(
    Or(x1, x2),
    Or(Not(x1), x3),
    Or(Not(x2), Not(x3))
)
```

# Solving the Formula

We create a solver and add the formula.

```
# Create solver
s = Solver()

# Add formula
s.add(phi)

# Check satisfiability
print(s.check())
```

## Output

```
sat
```

# Getting a Model

If the formula is satisfiable, Z3 can return one satisfying assignment.

```
if s.check() == sat:
    print(s.model())
```

Example output:

```
[x1 = False, x2 = True, x3 = False]
```

So one solution is

$$x_1 = \text{False}, \qquad x_2 = \text{True}, \qquad x_3 = \text{False}.$$

# Checking the Assignment

For

$$x_1 = \text{False}, \quad x_2 = \text{True}, \quad x_3 = \text{False},$$

we check each clause:

$$(x_1 \vee x_2) = (\text{False} \vee \text{True}) = \text{True}$$

$$(\neg x_1 \vee x_3) = (\text{True} \vee \text{False}) = \text{True}$$

$$(\neg x_2 \vee \neg x_3) = (\text{False} \vee \text{True}) = \text{True}$$

Therefore,

$$\varphi = \text{True}.$$

# An Unsatisfiable Example

Consider the formula

$$\psi = x_1 \wedge \neg x_1.$$

In Z3:

```
x1 = Bool('x1')

psi = And(x1, Not(x1))

s = Solver()
s.add(psi)

print(s.check())
```

The output is:

```
unsat
```

# SAT vs UNSAT

| Output | Meaning |
|---|---|
| `sat` | A satisfying assignment exists |
| `unsat` | No satisfying assignment exists |
| `unknown` | Z3 cannot determine the answer |

# Template for Solving SAT with Z3

```
from z3 import *

# Define variables
x1 = Bool('x1')
x2 = Bool('x2')

# Define formula
phi = And(
    Or(x1, x2),
    Or(Not(x1), x2)
)

# Create solver
s = Solver()
s.add(phi)

# Check satisfiability
if s.check() == sat:
    print("SAT")
    print(s.model())
else:
    print("UNSAT")
```

Output:

```
SAT
[x1 = False, x2 = True]
```

# Boolean Operations in Z3

In SAT problems, Boolean formulas are constructed using logical operations.

| Operation | Mathematical Notation | Z3 Syntax |
|---|---|---|
| Negation | $\neg x$ | `Not(x)` |
| Conjunction | $x \wedge y$ | `And(x, y)` |
| Disjunction | $x \vee y$ | `Or(x, y)` |
| Implication | $x \Rightarrow y$ | `Implies(x, y)` |
| Equivalence | $x \Leftrightarrow y$ | `x == y` |
| Exclusive OR | $x \oplus y$ | `Xor(x, y)` |

# Boolean Operations in Z3

```
from z3 import *

x = Bool('x')
y = Bool('y')
z = Bool('z')

phi = And(
    Or(x, y),           # x OR y
    Implies(x, z),      # if x then z
    Not(And(y, z))      # NOT (y AND z)
)

s = Solver()
s.add(phi)

print(s.check())
print(s.model())
```

# Boolean Operations in Z3

The output is:

```
sat
[x = True, y = False, z = True]
```

Therefore, one satisfying assignment is

$$x = \text{True}, \qquad y = \text{False}, \qquad z = \text{True}.$$

# Useful Shortcuts in Z3 Python

In Z3 Python, we can write constraints more compactly using shortcuts.

| Shortcut | Meaning |
|---|---|
| `Bools('x y z')` | create several Boolean variables |
| `Ints('x y z')` | create several integer variables |
| `And(L)` | conjunction of all formulas in list $L$ |
| `And(*L)` | unpack list $L$ as arguments of `And` |
| `Or(L)` | disjunction of all formulas in list $L$ |
| `Or(*L)` | unpack list $L$ as arguments of `Or` |

# Creating Many Variables

Instead of writing variables one by one:

```
x1 = Bool('x1')
x2 = Bool('x2')
x3 = Bool('x3')
```

# Creating Many Variables

Instead of writing variables one by one:

```
x1 = Bool('x1')
x2 = Bool('x2')
x3 = Bool('x3')
```

we can use:

```
x1, x2, x3 = Bools('x1 x2 x3')
```

# Creating Many Variables

Instead of writing variables one by one:

```
x1 = Bool('x1')
x2 = Bool('x2')
x3 = Bool('x3')
```

we can use:

```
x1, x2, x3 = Bools('x1 x2 x3')
```

For integer variables:

```
a, b, c = Ints('a b c')
```

# Using Lists of Constraints

Suppose we have a list of constraints:

```
x1, x2, x3 = Bools('x1 x2 x3')

L = [
    Or(x1, x2),
    Implies(x1, x3),
    Not(And(x2, x3))
]
```

# Using Lists of Constraints

Suppose we have a list of constraints:

```
x1, x2, x3 = Bools('x1 x2 x3')

L = [
    Or(x1, x2),
    Implies(x1, x3),
    Not(And(x2, x3))
]
```

Then we can add all constraints at once:

```
s = Solver()

s.add(And(L))
```

# Using Lists of Constraints

Suppose we have a list of constraints:

```
x1, x2, x3 = Bools('x1 x2 x3')

L = [
    Or(x1, x2),
    Implies(x1, x3),
    Not(And(x2, x3))
]
```

Then we can add all constraints at once:

```
s = Solver()

s.add(And(L))
```

This means:

```
And(L[0],L[1],L[2])
```

# The Meaning of `And(*L)`

In Python, `*L` means unpack the list $L$.
So if

```
L = [A, B, C]
```

then

```
And(*L)
```

is the same as

```
And(A, B, C)
```

Example:

```
s.add(And(*L))
```

This is especially useful when constraints are generated automatically.

# Generating Constraints Automatically

We can generate Boolean variables using a list comprehension.

```
x = [Bool(f'x{i}') for i in range(1, 6)]
```

# Generating Constraints Automatically

We can generate Boolean variables using a list comprehension.

```
x = [Bool(f'x{i}') for i in range(1, 6)]
```

This creates:

$$x_1, x_2, x_3, x_4, x_5.$$

# Generating Constraints Automatically

We can generate Boolean variables using a list comprehension.

```
x = [Bool(f'x{i}') for i in range(1, 6)]
```

This creates:

$$x_1, x_2, x_3, x_4, x_5.$$

Now suppose all variables must be true:

```
constraints = [x[i] for i in range(5)]
s = Solver()
s.add(And(*constraints))

print(s)
print(s.check())
print(s.model())
```

# Generating Constraints Automatically

We can generate Boolean variables using a list comprehension.

```
x = [Bool(f'x{i}') for i in range(1, 6)]
```

This creates:

$$x_1, x_2, x_3, x_4, x_5.$$

Now suppose all variables must be true:

```
constraints = [x[i] for i in range(5)]
s = Solver()
s.add(And(*constraints))

print(s)
print(s.check())
print(s.model())
```

The output is

```
[And(x1, x2, x3, x4, x5)]
sat
[x1 = True, x2 = True, x3 = True, x4 = True, x5 = True]
```

# Example: At Least One Variable is True

Let

$$x_1, x_2, x_3, x_4, x_5$$

be Boolean variables.

```
x = [Bool(f'x{i}') for i in range(1, 6)]
s = Solver()

# At least one variable is True
s.add(Or(*x))

print(s.check())
print(s.model())
```

The command

```
Or(*x)
```

means

$$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5.$$

# Some Useful Z3 Commands

Z3 provides several useful commands for modeling logical and combinatorial problems.

| Command | Purpose |
|---|---|
| `PbEq` | exactly $k$ Boolean variables are true |
| `PbLe` | at most $k$ Boolean variables are true |
| `PbGe` | at least $k$ Boolean variables are true |
| `If` | if-then-else expression |
| `Sum` | sum of arithmetic expressions |
| `Distinct` | all values are different |

# Pseudo-Boolean Constraints: PbEq

PbEq is useful for modeling "exactly $k$" constraints.

$$\texttt{PbEq}([(x_1, 1), (x_2, 1), (x_3, 1)], 1) \text{ means } x_1 + x_2 + x_3 = 1.$$

```
x1, x2, x3 = Bools('x1 x2 x3')
s = Solver()

# Exactly one variable is True
s.add(PbEq([(x1,1), (x2,1), (x3,1)], 1))

print(s.check())
print(s.model())
```

# Pseudo-Boolean Constraints: PbEq

PbEq is useful for modeling "exactly $k$" constraints.

$$\texttt{PbEq}([(x_1, 1), (x_2, 1), (x_3, 1)], 1) \text{ means } x_1 + x_2 + x_3 = 1.$$

```
x1, x2, x3 = Bools('x1 x2 x3')
s = Solver()

# Exactly one variable is True
s.add(PbEq([(x1,1), (x2,1), (x3,1)], 1))

print(s.check())
print(s.model())
```

# Pseudo-Boolean Constraints: PbEq

PbEq is useful for modeling "exactly $k$" constraints.

$$\texttt{PbEq}([(x_1, 1), (x_2, 1), (x_3, 1)], 1) \text{ means } x_1 + x_2 + x_3 = 1.$$

```
x1, x2, x3 = Bools('x1 x2 x3')
s = Solver()

# Exactly one variable is True
s.add(PbEq([(x1,1), (x2,1), (x3,1)], 1))

print(s.check())
print(s.model())
```

Possible output:

```
sat
[x1 = True, x2 = False, x3 = False]
```

# At Most and At Least: `PbLe` and `PbGe`

`PbLe` means "at most", while `PbGe` means "at least".

`PbLe`$([(x_1, 1), (x_2, 1), (x_3, 1)], 2)$ means $x_1 + x_2 + x_3 \leq 2$.

# At Most and At Least: PbLe and PbGe

PbLe means "at most", while PbGe means "at least".

PbLe$([(x_1, 1), (x_2, 1), (x_3, 1)], 2)$ means $x_1 + x_2 + x_3 \leq 2$.

```
x1, x2, x3 = Bools('x1 x2 x3')
s = Solver()

# At most two variables are True
s.add(PbLe([(x1,1), (x2,1), (x3,1)], 2))

# At least one variable is True
s.add(PbGe([(x1,1), (x2,1), (x3,1)], 1))

print(s.check())
print(s.model())
```

# At Most and At Least: PbLe and PbGe

PbLe means "at most", while PbGe means "at least".

$$\texttt{PbLe}([(x_1, 1), (x_2, 1), (x_3, 1)], 2) \text{ means } x_1 + x_2 + x_3 \leq 2.$$

```
x1, x2, x3 = Bools('x1 x2 x3')
s = Solver()

# At most two variables are True
s.add(PbLe([(x1,1), (x2,1), (x3,1)], 2))

# At least one variable is True
s.add(PbGe([(x1,1), (x2,1), (x3,1)], 1))

print(s.check())
print(s.model())
```

Possible output:

```
sat
[x1 = True, x2 = True, x3 = False]
```

# If-Then-Else Expression: `If`

In Z3, `If` works like an if-then-else expression.

$$\texttt{If}(p, a, b) = \begin{cases} a, & \text{if } p \text{ is true,} \\ b, & \text{if } p \text{ is false.} \end{cases}$$

# If-Then-Else Expression: `If`

In Z3, `If` works like an if-then-else expression.

$$\texttt{If}(p, a, b) = \begin{cases} a, & \text{if } p \text{ is true,} \\ b, & \text{if } p \text{ is false.} \end{cases}$$

```
x = Int('x')
y = Int('y')
s = Solver()

# y = 10 if x is positive, otherwise y = 0
s.add(y == If(x > 0, 10, 0))
s.add(x == 3)

print(s.check())
print(s.model())
```

# If-Then-Else Expression: `If`

In Z3, `If` works like an if-then-else expression.

$$\texttt{If}(p, a, b) = \begin{cases} a, & \text{if } p \text{ is true,} \\ b, & \text{if } p \text{ is false.} \end{cases}$$

```
x = Int('x')
y = Int('y')
s = Solver()

# y = 10 if x is positive, otherwise y = 0
s.add(y == If(x > 0, 10, 0))
s.add(x == 3)

print(s.check())
print(s.model())
```

Output:

```
sat
[x = 3, y = 10]
```

# Using `If` with Boolean Variables

The command `If` can also return a Boolean expression.

$$\texttt{If}(p, q, r) = \begin{cases} q, & \text{if } p = \text{True}, \\ r, & \text{if } p = \text{False}. \end{cases}$$

# Using `If` with Boolean Variables

The command `If` can also return a Boolean expression.

$$\texttt{If}(p, q, r) = \begin{cases} q, & \text{if } p = \text{True}, \\ r, & \text{if } p = \text{False}. \end{cases}$$

```
from z3 import *

p, q, r = Bools('p q r')
s = Solver()

# If p is True, then q must be True.
# If p is False, then r must be True.
s.add(If(p, q, r))

print(s.check())
print(s.model())
```

# Using `If` with Boolean Variables

The command `If` can also return a Boolean expression.

$$\texttt{If}(p, q, r) = \begin{cases} q, & \text{if } p = \text{True}, \\ r, & \text{if } p = \text{False}. \end{cases}$$

```
from z3 import *

p, q, r = Bools('p q r')
s = Solver()

# If p is True, then q must be True.
# If p is False, then r must be True.
s.add(If(p, q, r))

print(s.check())
print(s.model())
```

Output:

```
sat
[p = True, q = True]
```

# Using `If` to Count True Variables

We can use `If` to convert Boolean values into integers.

$$\texttt{If}(x,\ 1,\ 0) = \begin{cases} 1, & x = \text{True}, \\ 0, & x = \text{False}. \end{cases}$$

# Using `If` to Count True Variables

We can use `If` to convert Boolean values into integers.

$$\texttt{If}(x,\ 1,\ 0) = \begin{cases} 1, & x = \text{True}, \\ 0, & x = \text{False}. \end{cases}$$

```
x1, x2, x3 = Bools('x1 x2 x3')
count_true = Sum(
    If(x1, 1, 0),
    If(x2, 1, 0),
    If(x3, 1, 0)
)
s = Solver()

# Exactly two variables are True
s.add(count_true == 2)
print(s.check())
print(s.model())
```

# Using `If` to Count True Variables

We can use `If` to convert Boolean values into integers.

$$\texttt{If}(x,\ 1,\ 0) = \begin{cases} 1, & x = \text{True}, \\ 0, & x = \text{False}. \end{cases}$$

```
x1, x2, x3 = Bools('x1 x2 x3')
count_true = Sum(
    If(x1, 1, 0),
    If(x2, 1, 0),
    If(x3, 1, 0)
)
s = Solver()

# Exactly two variables are True
s.add(count_true == 2)
print(s.check())
print(s.model())
```

Possible output:

```
sat
[x1 = True, x2 = True, x3 = False]
```

# Using `Distinct` with Boolean Variables

The command `Distinct` means that all variables must have different values. For Boolean variables, there are only two possible values:

$$\text{True} \quad \text{and} \quad \text{False}.$$

Therefore, at most two Boolean variables can be pairwise distinct.

```
from z3 import *

x, y = Bools('x y')
s = Solver()

# x and y must have different truth values
s.add(Distinct(x, y))

print(s.check())
print(s.model())
```

Possible output:

```
sat
[x = False, y = True]
```

# Three Boolean Variables with `Distinct`

If we use `Distinct` for three Boolean variables, the problem becomes impossible.

```
from z3 import *

x, y, z = Bools('x y z')
s = Solver()

# Impossible: only True and False are available
s.add(Distinct(x, y, z))
print(s.check())
```

Output:

```
unsat
```

## Important Observation

For Boolean variables, `Distinct(x,y,z)` is unsatisfiable because three variables cannot all be different when there are only two truth values.

# Why Use Z3 for SAT Problems?

- We can translate logical problems into constraints.
- Z3 automatically searches for satisfying assignments.
- It is useful for puzzles, scheduling, verification, and combinatorial problems.
- Google Colab makes it easy to run Python and Z3 without local installation.

# Solving the Festival Planning Problem

A university is organizing a *one-day innovation festival*. To run the event, the committee must choose exactly one venue, one keynote speaker, and one workshop theme from the following options:

(i) Venue: Main Hall ($v_1$), Outdoor Plaza ($v_2$), and Auditorium ($v_3$)

(ii) Keynote Speaker: Entrepreneur ($s_1$), Scientist ($s_2$), and Government Official ($s_3$)

(iii) Workshop Theme: Artificial Intelligence ($w_1$), Sustainability ($w_2$), and Cybersecurity ($w_3$)

However, the committee faces the following constraints:

(i) If the event is held in the Outdoor Plaza, then the Government Official cannot be invited.

(ii) The festival must feature either the Scientist or the Cybersecurity workshop.

(iii) If the workshop theme is Sustainability, then it cannot be held in the Main Hall.

(iv) The Entrepreneur cannot be paired with the Artificial Intelligence workshop.

(v) If the event is held in the Auditorium, then the workshop must be Artificial Intelligence or Cybersecurity.

(a) Express each constraint as a clause using the variables above.

(b) Determine whether the planning problem is satisfiable. If it is, give one satisfying assignment for all variables.

# Solving the Festival Planning Problem

```
from z3 import *
# Variables
v1, v2, v3 = Bools('v1 v2 v3')    # venues
s1, s2, s3 = Bools('s1 s2 s3')    # speakers
w1, w2, w3 = Bools('w1 w2 w3')    # workshops

s = Solver()

# Exactly one venue, one speaker, one workshop
s.add(PbEq([(v1,1), (v2,1), (v3,1)], 1))
s.add(PbEq([(s1,1), (s2,1), (s3,1)], 1))
s.add(PbEq([(w1,1), (w2,1), (w3,1)], 1))

# Constraints
s.add(Implies(v2, Not(s3)))      # Outdoor Plaza -> not Govt Official
s.add(Or(s2, w3))                # Scientist or Cybersecurity
s.add(Implies(w2, Not(v1)))      # Sustainability -> not Main Hall
s.add(Not(And(s1, w1)))          # not(Entrepreneur and AI)
s.add(Implies(v3, Or(w1, w3)))  # Auditorium -> AI or Cybersecurity

print(s.check())
print(s.model())
```

# Interpreting the Z3 Output

One possible output is:

```
sat
[v1 = False,
 v2 = False,
 v3 = True,
 s1 = False,
 s2 = True,
 s3 = False,
 w1 = True,
 w2 = False,
 w3 = False]
```

Hence one satisfying assignment is:

$$v_3 = \text{True}, \qquad s_2 = \text{True}, \qquad w_1 = \text{True}.$$

## Conclusion

The planning problem is satisfiable. One valid plan is:

Auditorium, Scientist, Artificial Intelligence.